

팍리스 실무형 일경험 프로그램

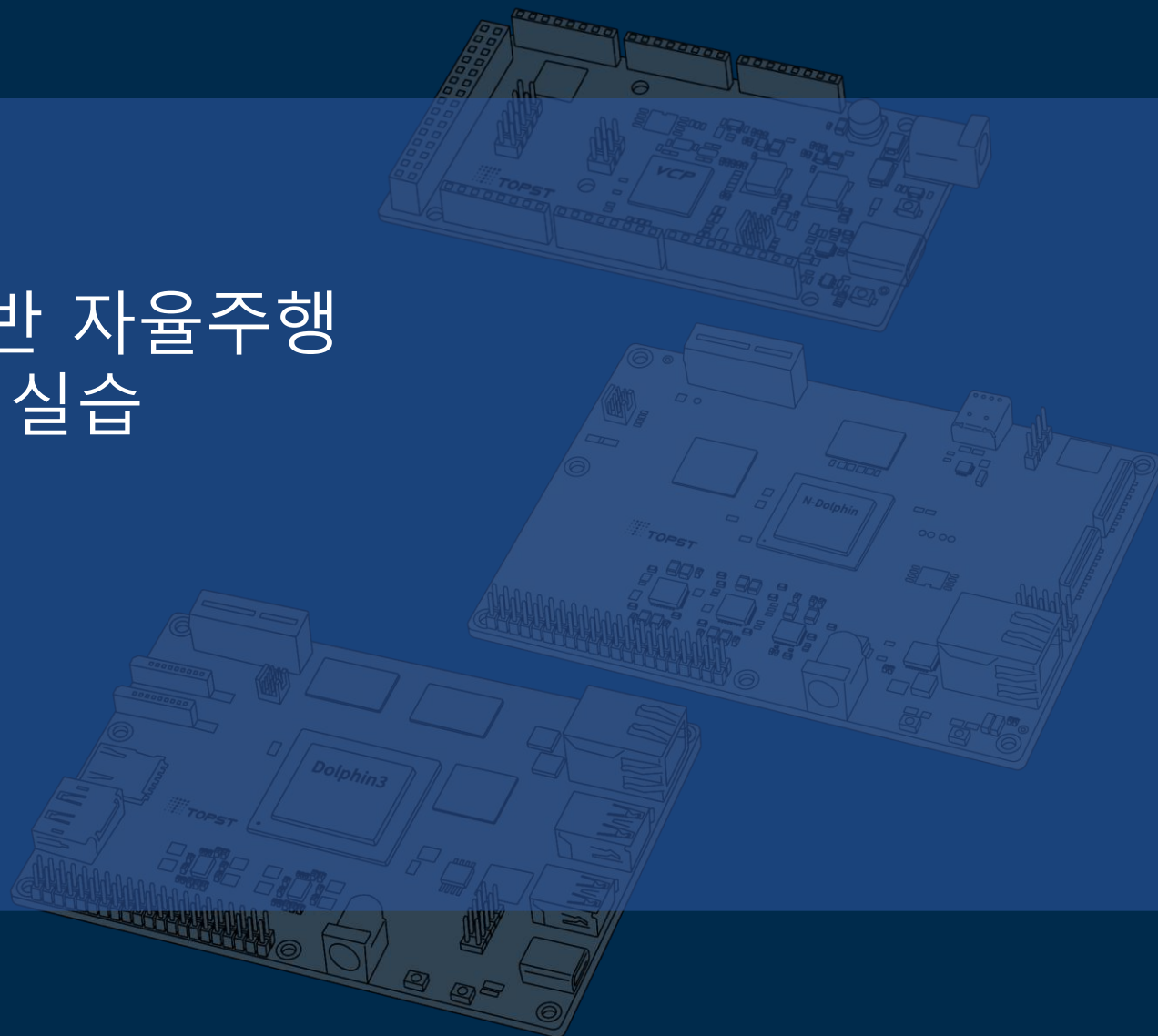
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



1일차

03 Yocto의 이해

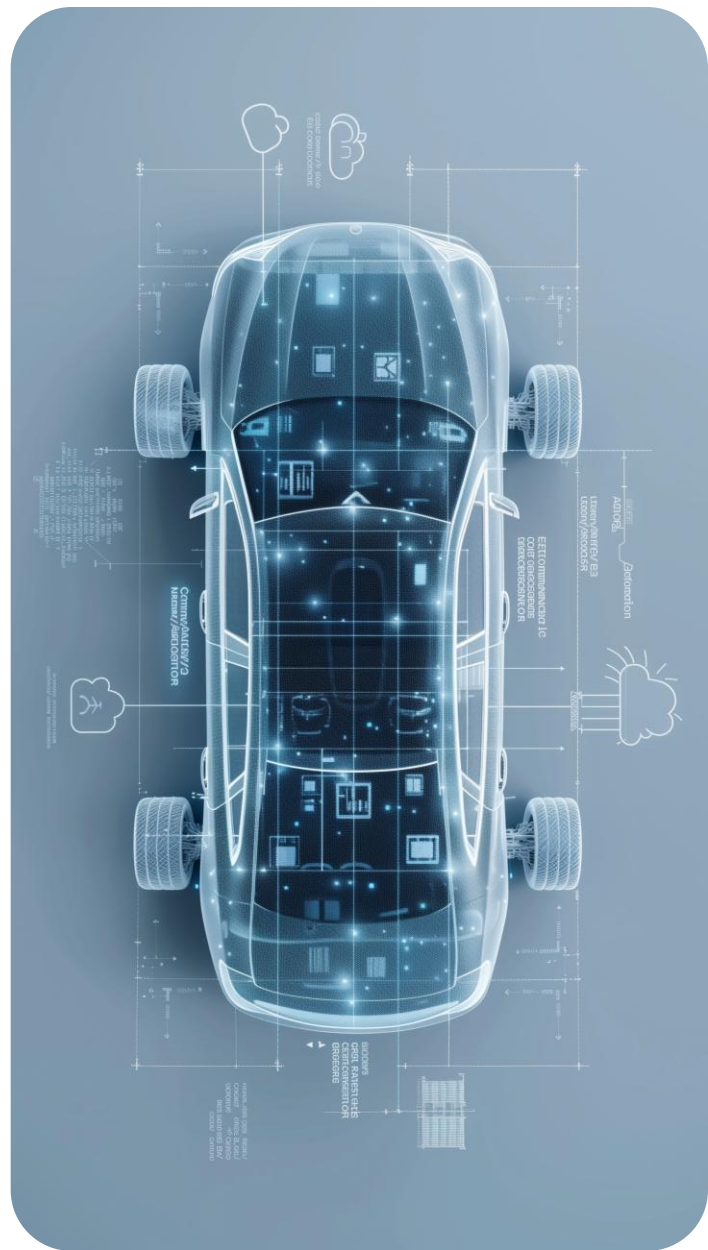
Telechips TOPST



목차

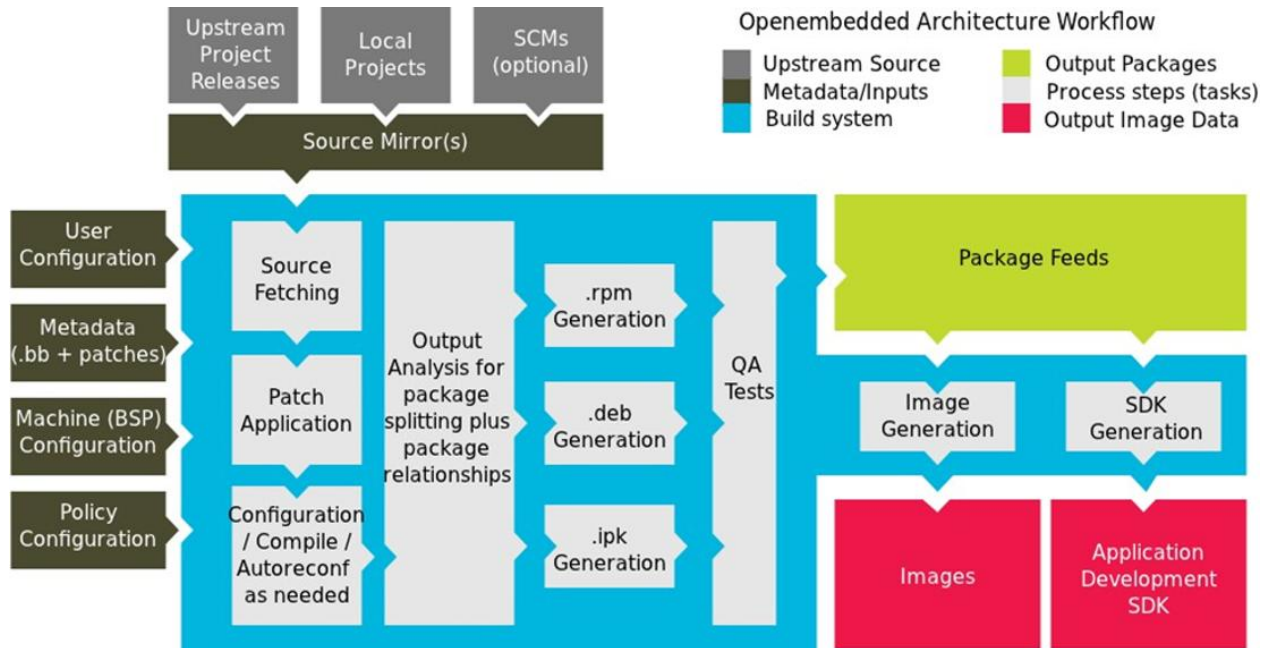
Table of Contents

Yocto 소개	01
Bitbake와 Recipe	02
Poky와 open-embedded	03
Hello world recipe	04



Yocto 소개

- Embedded Linux를 빌드하고 배포하기 위한 Framework
 - User-defined Linux platform (Kernel, boot loader, rootfs)
 - 다양한 Hardware, architecture, devices, applications
- Bitbake, Recipe 기반 빌드 시스템
- 유지보수의 편의성 제공



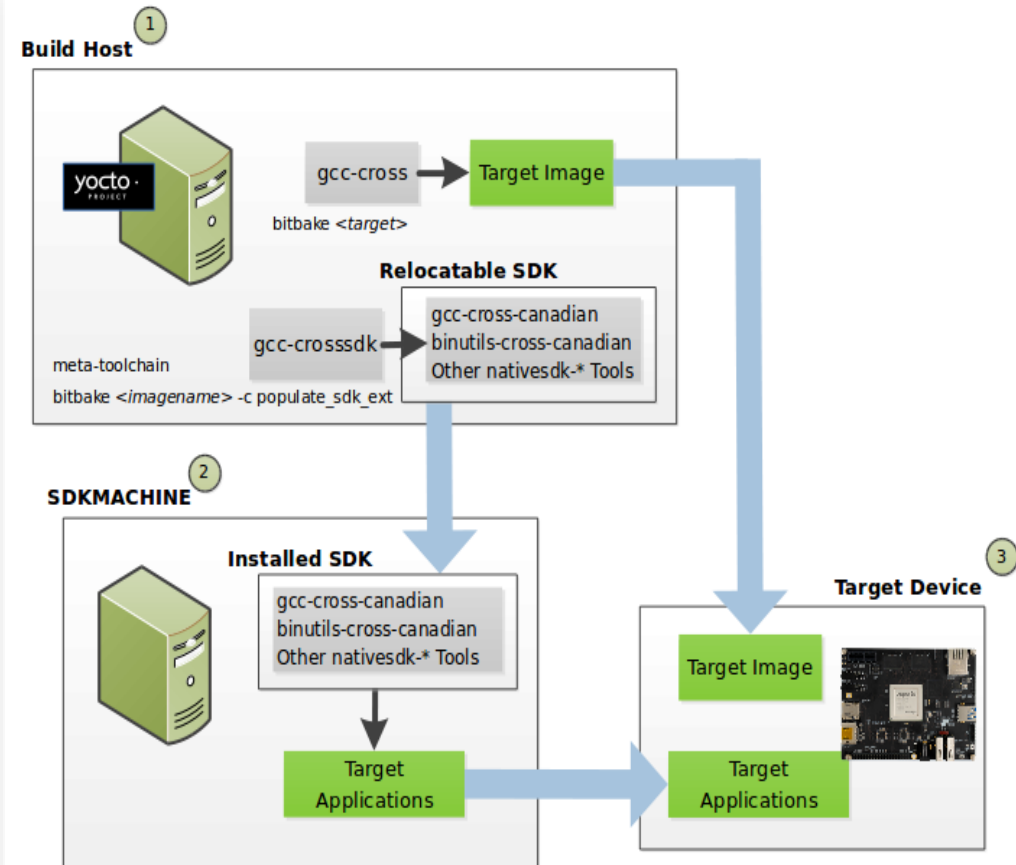
Yocto 소개

- 임베디드 시스템에 맞는 리눅스 커널과 시스템 파일들을 구성 및 빌드하고
이를 배포하기 위한 오픈소스 프로젝트
- **장점**
 - Target 보드의 특성에 맞는 환경 설정 가능
 - 의존성에 따른 소프트웨어와 하드웨어 구조 설정 가능
 - 변경 사항에 대해 재사용이 가능
 - 커스텀 리눅스 설치의 어려움 해결
- **단점**
 - 배우기 어려움
 - 빌드 시간이 오래 걸림

빌드 에러에 대한 대처가 용이하지 않음

Yocto 구성

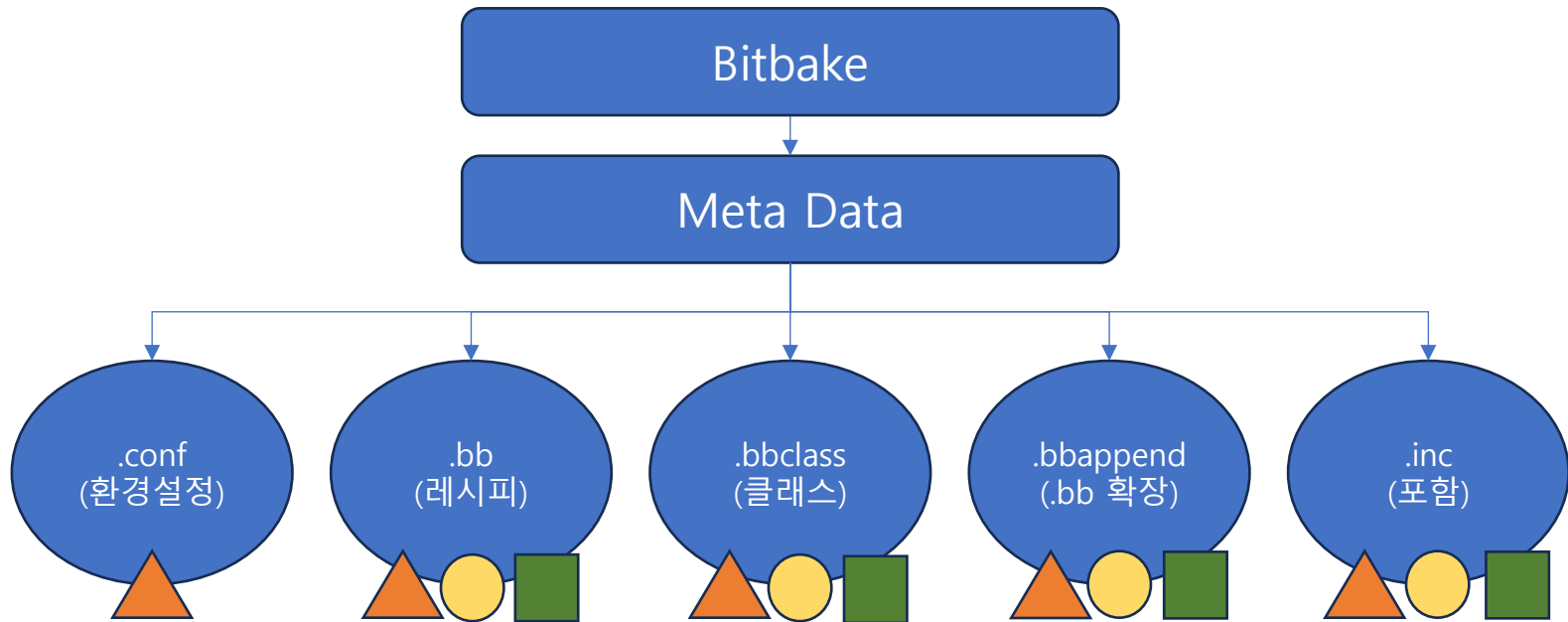
- Yocto Build host
 - Kernel, Kernel Modules
 - BootLoader
 - Rootfs, etc
- SDK Toolkit
- Target Device



Bitbake와 Meta 데이터

- Bitbake
 - Python으로 작성된 프로그램
 - Meta 데이터를 해석해 필요한 작업을 분석하고 실행
- Meta 데이터
 - Package 및 Image 빌드를 위한 설정 내용
 - 변수, 함수, 태스크 등을 명세
 - 빌드를 위한 의존성 기술
- 레이어와 계층 구조
 - Meta 데이터의 복잡성을 세분화
- Package와 Image recipe
 - 시스템 구성요소를 빌드하는 Package recipe
 - Package 들을 모아 파일 시스템을 구성하는 Image recipe

Bitbake와 Meta 데이터



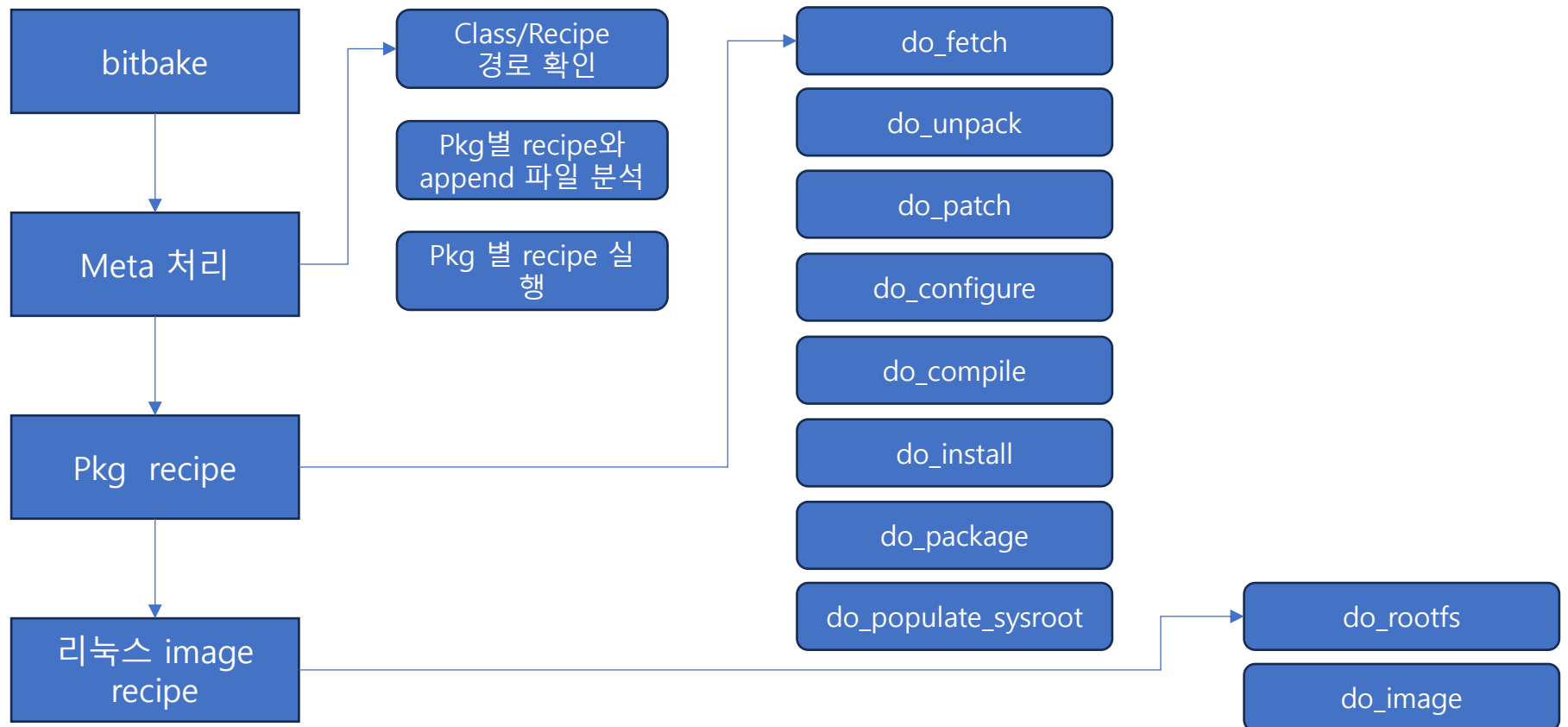
▲ 변수 선언

● 함수 선언

■ 태스크 선언

Bitbake의 이해

• Bitbake가 실행되는 과정



Bitbake의 이해

- Yocto TASK의 이해
 - `do_fetch` : 지정된 위치에서 소스코드를 복사 또는 clone
 - `do_unpack` : 필요하면 압축파일을 해제
 - `do_patch` : 필요하면 patch 파일을 적용
 - `do_configure` : 빌드 환경 설정
 - `do_compile` : 소스 컴파일 하거나 빌드 과정을 위한 스크립트 실행
 - `do_install` : `do_compile`이 완료된 산출물을 결과물로 저장
 - `do_package` : 결과물을 package 로 생성
 - `do_populate_sysroot` : `do_install` 된 산출물을 system root 폴더로 복사
 - `do_rootfs` : system root 폴더 내용을 사용해 rootfs 생성
 - `do_image` : 생성된 rootfs를 ext4 등으로 이미지 생성
- Recipe에 명시 되지 않은 task는 inherit class를 통해 상속 가능
 - Ex) `cmake.bbclass` : `do_configure`, `do_compile` task에 `cmake_do_configure`, `cmake_do_compile` 이 연결되어 있음

Bitbake의 이해

- 레이어와 하위 구조
 - Meta 레이어는 개념상 기능군을 의미
 - 다양한 기능군들이 모여 전체 기능을 구성
 - 여러 Meta 데이터 군 형성 -> 레이어
 - 아래는 TOPST D3-G 제품의 meta layer

```
kypark@TOPST-Build-SVR:~/Work/d3g-gitlab-v1.2.0/build/d3-g-topst-main$ bitbake-layers show-layers
```

NOTE: Starting bitbake server...

layer	path	priority
meta	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta	5
meta-poky	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-poky	5
meta-arm-toolchain	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-arm/meta-arm-toolchain	5
meta-oe	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-openembedded/meta-oe	5
meta-python	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-openembedded/meta-python	5
meta-multimedia	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-openembedded/meta-multimedia	5
meta-webkit	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-webkit	7
meta-topst-bsp	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-topst-bsp	8
meta-topst	/home/kypark/Work/d3g-gitlab-v1.2.0/poky/meta-topst	11

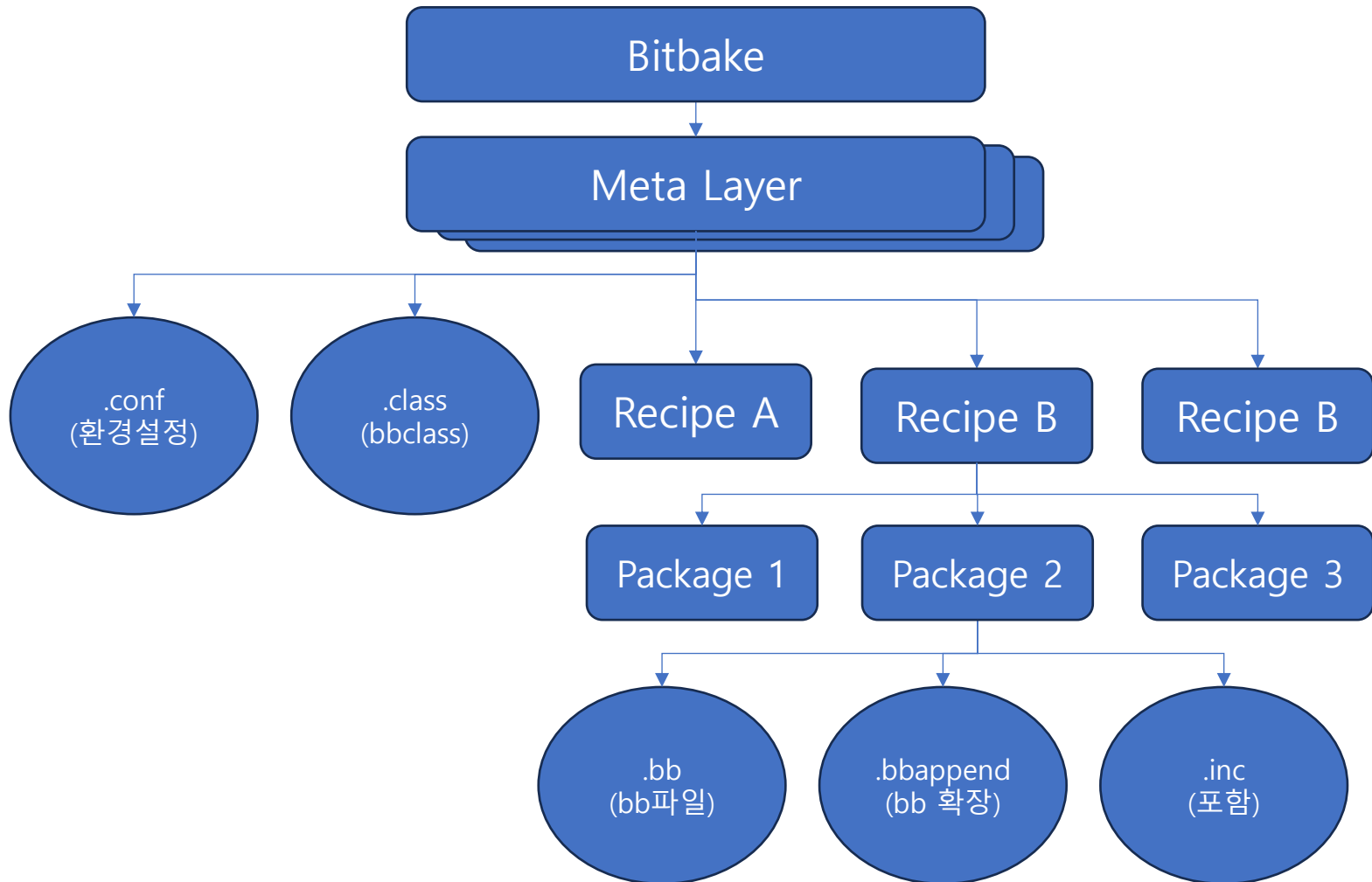
```
kypark@TOPST-Build-SVR:~/Work/d3g-gitlab-v1.2.0/build/d3-g-topst-main$
```

Bitbake의 이해

- 레이어와 하위 구조
 - 하나의 meta layer는 세분화 된 하위 기능군들의 집합체
 - 여러 하위 기능군 형
 - 아래는 openembedded.org의 예

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta$ ls -al
total 96
drwxrwxr-x 20 kypark kypark 4096 Oct 13 11:14 .
drwxrwxr-x 14 kypark kypark 4096 Nov  4 09:00 ..
drwxrwxr-x  2 kypark kypark 12288 Oct 13 14:37 classes
drwxrwxr-x  6 kypark kypark 4096 Oct 13 11:14 conf
-rw-rw-r--  5 kypark kypark 1035 Oct 13 11:13 COPYING.MIT
drwxrwxr-x  4 kypark kypark 4096 Oct 13 11:14 files
drwxrwxr-x  6 kypark kypark 4096 Oct 13 11:46 lib
drwxrwxr-x 21 kypark kypark 4096 Oct 13 11:14 recipes-bsp
drwxrwxr-x 26 kypark kypark 4096 Oct 13 11:14 recipes-connectivity
drwxrwxr-x 44 kypark kypark 4096 Oct 13 11:14 recipes-core
drwxrwxr-x 97 kypark kypark 4096 Oct 13 11:14 recipes-devtools
drwxrwxr-x  3 kypark kypark 4096 Oct 13 11:14 recipes-example
drwxrwxr-x 88 kypark kypark 4096 Oct 13 11:14 recipes-extended
drwxrwxr-x 19 kypark kypark 4096 Oct 13 11:14 recipes-gnome
drwxrwxr-x 49 kypark kypark 4096 Oct 13 11:14 recipes-graphics
drwxrwxr-x 18 kypark kypark 4096 Oct 13 11:14 recipes-kernel
drwxrwxr-x 23 kypark kypark 4096 Oct 13 11:14 recipes-multimedia
drwxrwxr-x  4 kypark kypark 4096 Oct 13 11:14 recipes-rt
drwxrwxr-x 21 kypark kypark 4096 Oct 13 11:14 recipes-sato
drwxrwxr-x 86 kypark kypark 4096 Oct 13 11:14 recipes-support
-rw-rw-r--  1 kypark kypark 1209 Oct 13 11:13 recipes.txt
drwxrwxr-x  2 kypark kypark 4096 Oct 13 11:14 site
```

Bitbake와 Meta 레이어의 계층 구조



Bitbake의 레시피 작성

- 레시피 (.bb) 의 작성과 문법

```
DESCRIPTION="it's a simple hello world"
LICENSE="CLOSED"

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI = "file://helloworld.tar.gz"
S = "${WORKDIR}/helloworld"

do_compile() {
    ${CC} helloworld.c ${LDFLAGS} -o helloworld
}

do_install() {
    install -d ${D}${bindir}
    install -m 0755 ${B}/helloworld ${D}${bindir}/helloworld
}
```

Poky와 OpenEmbedded

- Poky의 이해
 - Yocto 프로젝트를 위한 참조 배포
 - 커스텀 리눅스 빌드를 위해 필수인 Tool 집합
- Poky 디렉터리 구조

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky$ ls -al
total 120
drwxrwxr-x 14 kypark kypark 4096 Nov  4 09:00 .
drwxrwxr-x  4 kypark kypark 4096 Oct 13 18:02 ..
drwxrwxr-x  6 kypark kypark 4096 Oct 13 11:14 bitbake
-rw-rw-r--  1 kypark kypark 3424 Nov  4 08:54 bitbake-cookerdaemon.log
drwxrwxr-x  7 kypark kypark 4096 Nov  4 11:51 build
drwxrwxr-x  4 kypark kypark 4096 Nov  4 08:54 contrib
drwxrwxr-x 21 kypark kypark 4096 Oct 13 11:14 documentation
drwxrwxr-x  8 kypark kypark 4096 Nov  4 11:07 .git
-rw-rw-r--  1 kypark kypark  702 Oct 13 11:14 .gitignore
-rw-rw-r--  1 kypark kypark  834 Oct 13 11:13 LICENSE
-rw-rw-r--  1 kypark kypark 15394 Oct 13 11:13 LICENSE.GPL-2.0-only
-rw-rw-r--  1 kypark kypark  1286 Oct 13 11:13 LICENSE.MIT
-rw-rw-r--  1 kypark kypark  2202 Oct 13 11:14 MAINTAINERS.md
-rw-rw-r--  1 kypark kypark  1222 Oct 13 11:14 Makefile
-rw-rw-r--  1 kypark kypark   244 Oct 13 11:13 MEMORIAM
drwxrwxr-x 20 kypark kypark 4096 Oct 13 11:14 meta
drwxrwxr-x  4 kypark kypark 4096 Oct 14 14:33 meta-helloworld
drwxrwxr-x  5 kypark kypark 4096 Oct 13 11:14 meta-poky
drwxrwxr-x  9 kypark kypark 4096 Oct 13 11:14 meta-selftest
drwxrwxr-x  8 kypark kypark 4096 Oct 13 11:14 meta-skeleton
drwxrwxr-x  8 kypark kypark 4096 Oct 13 11:14 meta-yocto-bsp
-rwxrwxr-x  1 kypark kypark 1297 Oct 13 11:14 oe-init-build-env
lrwxrwxrwx  1 kypark kypark    33 Oct 13 11:13 README.hardware.md -> meta-yocto-bsp/README.hardware.md
lrwxrwxrwx  1 kypark kypark    14 Oct 13 11:13 README.md -> README.poky.md
-rw-rw-r--  1 kypark kypark  791 Oct 13 11:14 README.OE-Core.md
lrwxrwxrwx  1 kypark kypark    24 Oct 13 11:13 README.poky.md -> meta-poky/README.poky.md
-rw-rw-r--  1 kypark kypark   529 Oct 13 11:14 README.qemu.md
drwxrwxr-x 10 kypark kypark 4096 Oct 13 11:14 scripts
-rw-rw-r--  1 kypark kypark  1177 Oct 13 11:14 SECURITY.md
-rw-rw-r--  1 kypark kypark    65 Oct 13 11:14 .templateconf
```

Poky를 이용한 core-image 빌드

- Openembedded project를 git clone
 - Git clone git://git.openembedded.org/poky
 - Git checkout kirkstone
- 환경 설정
 - source oe-init-build-env
 - Bitbake core-image-minimal

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky$ source oe-init-build-env
```

```
### Shell environment set up for builds. ###
```

You can now run 'bitbake <target>'

Common targets are:

```
core-image-minimal
core-image-full-cmdline
core-image-sato
core-image-weston
meta-toolchain
meta-ide-support
```

You can also run generated qemu images with a command like 'runqemu qemux86'

Other commonly useful commands are:

- 'devtool' and 'recipetool' handle common recipe tasks
- 'bitbake-layers' handles common layer tasks
- 'oe-pkgdata-util' handles common target package tasks

Poky를 이용한 core-image 빌드

- Custom linux 빌드를 제공
 - 어떤 속성의 Linux를 빌드 하나 ?
 - build/conf/local.conf 에 답이 있음 (MACHINE 변수)

```
..  
#MACHINE ?= "qemuarm"  
#MACHINE ?= "qemuarm64"  
#MACHINE ?= "qemumips"  
#MACHINE ?= "qemumips64"  
#MACHINE ?= "qemuppc"  
#MACHINE ?= "qemux86"  
#MACHINE ?= "qemux86-64"  
#  
# There are also the following hardware board target machines included for  
# demonstration purposes:  
#  
#MACHINE ?= "beaglebone-yocto"  
#MACHINE ?= "genericx86"  
#MACHINE ?= "genericx86-64"  
#MACHINE ?= "edgerouter"  
#  
# This sets the default machine to be qemux86-64 if no other machine is selected:  
MACHINE ??= "qemux86-64"
```


Poky를 이용한 core-image 빌드

- Custom linux 빌드를 제공
 - MACHINE 변수에 해당하는 system conf 파일을 찾아 설정
 - 대개 oe-core 의 conf/machine 에서 해당 파일을 찾음
 - Kernel 및 System 환경에 필요한 설정 내용이 위치해 있음
 - MACHINE 설정의 conf 파일은 한번만 선택되며, 여러 layer에 존재할 경우 layer 우선 순위에 따라 선택

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta/conf/machine$ ls -al
total 56
drwxrwxr-x  3 kypark kypark 4096 Oct 13 11:14 .
drwxrwxr-x  6 kypark kypark 4096 Oct 13 11:14 ..
drwxrwxr-x 10 kypark kypark 4096 Oct 13 11:14 include
-rw-rw-r--  1 kypark kypark 1313 Oct 13 11:14 qemuarm64.conf
-rw-rw-r--  1 kypark kypark 1393 Oct 13 11:14 qemuarm.conf
-rw-rw-r--  1 kypark kypark  737 Oct 13 11:14 qemuarmv5.conf
-rw-rw-r--  1 kypark kypark  486 Oct 13 11:13 qemumips64.conf
-rw-rw-r--  1 kypark kypark  468 Oct 13 11:13 qemumips.conf
-rw-rw-r--  1 kypark kypark  614 Oct 13 11:14 qemuppc64.conf
-rw-rw-r--  1 kypark kypark  586 Oct 13 11:13 qemuppc.conf
-rw-rw-r--  1 kypark kypark  400 Oct 13 11:14 qemuriscv32.conf
-rw-rw-r--  1 kypark kypark  295 Oct 13 11:13 qemuriscv64.conf
-rw-rw-r--  1 kypark kypark 1643 Oct 13 11:14 qemux86-64.conf
-rw-rw-r--  1 kypark kypark 1397 Oct 13 11:14 qemux86.conf
```

Poky를 이용한 core-image 빌드

- Custom linux 빌드를 제공
 - Machine conf 파일에 빌드를 위한 linux recipe를 명시
 - Qemux86-64.conf > require qemu.inc
 - virtual/kernel 이 빌드하고자 하는 linux를 가리키며, linux-yocto recipe가 이에 해당

```
XSERVER ?= "xserver-xorg \
    ${@bb.utils.contains('DISTRO_FEATURES', 'opengl', 'mesa-driver-swrast xserver'
    xf86-video-fbdev \
    "
```

```
MACHINE_FEATURES = "alsa bluetooth usb gadget screen vfat"
```

```
MACHINEOVERRIDES =. "qemuall:"
```

```
IMAGE_FSTYPES += "tar.bz2 ext4"
```

```
# Don't include kernels in standard images
RDEPENDS:${KERNEL_PACKAGE_NAME}-base = ""
```

```
# Use a common kernel recipe for all QEMU machines
```

```
PREFERRED_PROVIDER_virtual/kernel ??= "linux-yocto"
```

```
EXTRA_IMAGEDEPENDS += "qemu-system-native qemu-helper-native:do_addto_recipe_sysroot"
```

```
# Provide the nfs server kernel module for all qemu images
```

```
KERNEL_FEATURES:append:pn-linux-yocto = " features/nfsd/nfsd-enable.scc"
```

```
KERNEL_FEATURES:append:pn-linux-yocto-rt = " features/nfsd/nfsd-enable.scc"
```

Poky를 이용한 core-image 빌드

- Custom linux 빌드를 제공
 - Meta 폴더의 recipe-kernel 에 linux-yocto recipe들이 존재
 - 별도 package 버전 명세가 없을 경우, 가장 높은 5.15 package 버전을 선택

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta/recipes-kernel/linux$ ls -al
total 528
drwxrwxr-x  3 kypark kypark  4096 Oct 13 11:14 .
drwxrwxr-x 18 kypark kypark  4096 Oct 13 11:14 ..
-rw-rw-r--  1 kypark kypark 204750 Oct 13 11:14 cve-exclusion_5.10.inc
-rw-rw-r--  1 kypark kypark 261096 Oct 13 11:14 cve-exclusion_5.15.inc
-rw-rw-r--  1 kypark kypark   383 Oct 13 11:14 cve-exclusion.inc
-rwxrwxr-x  1 kypark kypark  3543 Oct 13 11:14 generate-cve-exclusions.py
-rw-rw-r--  1 kypark kypark 15628 Oct 13 11:14 kernel-devsrc.bb
drwxrwxr-x  2 kypark kypark  4096 Oct 13 11:13 linux-dummy
-rw-rw-r--  1 kypark kypark  1304 Oct 13 11:14 linux-dummy.bb
-rw-rw-r--  1 kypark kypark  2961 Oct 13 11:14 linux-yocto_5.10.bb
-rw-rw-r--  1 kypark kypark  3494 Oct 13 11:14 linux-yocto_5.15.bb
-rw-rw-r--  1 kypark kypark  3329 Oct 13 11:14 linux-yocto-dev.bb
-rw-rw-r--  1 kypark kypark  3018 Oct 13 11:14 linux-yocto.inc
-rw-rw-r--  1 kypark kypark  2020 Oct 13 11:14 linux-yocto-rt_5.10.bb
-rw-rw-r--  1 kypark kypark  2020 Oct 13 11:14 linux-yocto-rt_5.15.bb
-rw-rw-r--  1 kypark kypark  1072 Oct 13 11:14 linux-yocto-tiny_5.10.bb
-rw-rw-r--  1 kypark kypark   950 Oct 13 11:14 linux-yocto-tiny_5.15.bb
```

Hello World

- META 레이어를 만들고 추가

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta-helloworld$ tree
```

```
.
├── conf
│   └── layer.conf
├── recipes-hello
│   └── helloworld
│       ├── files
│       │   ├── helloworld
│       │   │   └── helloworld.c
│       │   └── helloworld.tar.gz
│       └── helloworld_1.0.bb
```

- Layer.conf 생성

- Layer를 만들면 conf/layer.conf 파일 필수

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta-helloworld/conf$ cat layer.conf
```

```
# We have a conf and classes directory, add to BBPATH
```

```
BBPATH .= ":${LAYERDIR}"
```

```
# We have recipes-* directories, add to BBFILES
```

```
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
           ${LAYERDIR}/recipes-*/*/*.bbappend"
```

```
BBFILE_COLLECTIONS += "helloworld"
```

```
BBFILE_PATTERN_helloworld = "^${LAYERDIR}/"
```

```
BBFILE_PRIORITY_helloworld = "10"
```

```
LAYERVERSION_helloworld = "4"
```

```
LAYERSERIES_COMPAT_helloworld = "kirkstone"
```

Hello World

- 간단한 helloworld.c 의 모양

```
#include <stdio.h>

int main()
{
    printf("----hellow world !!! \n");
    return 0;
}
```

- Files 폴더내 소스코드와 압축파일
 - Helloworld.tar.gz를 통해 bb 파일내 SRC_URI로 접근하게 한다.
 - tar -zcvf helloworld.tar.gz helloworld

```
cympark@TOPST-Build-SVR:~/Work/yocto-study/poky/meta-helloworld/recipes-hello/helloworld/files$ tree
```

```
├── helloworld
│   └── helloworld.c
└── helloworld.tar.gz
```

Hello World

- Layer를 추가 한다.
 - Bitbake-layers add-layer meta-helloworld
 - Bitbake-layers show-layers

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/build$ bitbake-layers add-layer ../meta-helloworld
NOTE: Starting bitbake server...
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/build$ bitbake-layers show-layers
NOTE: Starting bitbake server...
layer                path                                     priority
=====
meta                  /home/kypark/Work/yocto-study/poky/meta  5
meta-poky             /home/kypark/Work/yocto-study/poky/meta-poky  5
meta-yocto-bsp        /home/kypark/Work/yocto-study/poky/meta-yocto-bsp  5
meta-helloworld       /home/kypark/Work/yocto-study/poky/meta-helloworld  10
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/build$
```

- Helloworld를 빌드한다.
 - Bitbake helloworld

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/build$ bitbake helloworld
Loading cache: 100% |
Loaded 0 entries from dependency cache.
Parsing recipes: 100% |#####
Parsing of 884 .bb files complete (0 cached, 884 parsed). 1645 targets, 45 skipped, 0 masked, 0 errors
NOTE: Resolving any missing task queue dependencies
```

Build Configuration:

Hello World

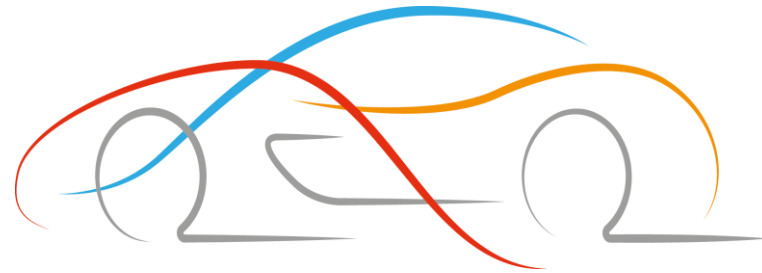
- Rootfs 에 포함되게 만들려면 package 설치를 명시해야 하며, conf/local.conf 나 image recipe 내 IMAGE_INSTALL 변수에 추가
 - 아래는 poky/build/conf/local.conf 내 추가

```
# CONF_VERSION is increased each time build/conf/ changes incomp
# track the version of this file when it was generated. This can
# this doesn't mean anything to you.
CONF_VERSION = "2"
```

```
IMAGE_INSTALL:append = " helloworld"
```

- Bitbake core-image-minimal 로 다시 이미지 빌드한 후, rootfs 내용 확인
 - 아래는 do_rootfs 후, do_image 를 진행하는 기본 폴더
 - 아래 rootfs 폴더에 필요한 package가 적재되어 있는 것을 확인

```
kypark@TOPST-Build-SVR:~/Work/yocto-study/poky/build$ ls -al tmp/work/qemux86_64-poky-linux/core-image-minimal/1.0-r0/rootfs/usr/bin/helloworld
-rwxr-xr-x 1 kypark kypark 14376 Mar  9 2018 tmp/work/qemux86_64-poky-linux/core-image-minimal/1.0-r0/rootfs/usr/bin/helloworld
```



Thank You